

# Mastering Policy Gradients & Variance Reduction

Complete Mathematical & Algorithmic Foundations: The Likelihood Ratio Trick, The Log-Derivative Identity, Causality & Reward-to-Go, The EGLP Lemma, Optimal Baselines, and Production PyTorch REINFORCE

**Instructor:** Prof. Sergey Levine (UC Berkeley) **Curriculum:** Berkeley CS285 + Achiam (Spinning Up) + Sutton & Barto **Scope:** General Continuous Policy Optimization & Robotics Slicing

## 0. The Executive Mental Map: Why Does Lecture 5 Exist?

In supervised deep learning, optimization is straightforward: compute loss  $\mathcal{L}(\theta) = \|f_\theta(x) - y\|^2$  and backpropagate gradients  $\nabla_\theta \mathcal{L}$  directly through network layers.

**The Central Dilemma of Robotics:** In physical control, between the policy's motor outputs and the scalar reward lies **the physical universe**: contact mechanics, non-linear friction, fluid dynamics, and biological tissue fracture. You cannot compute  $\frac{\partial \text{Tissue Fracture}}{\partial \text{Motor Current}}$  through analytical backpropagation! The real world is non-differentiable.

**Lecture 5 introduces the Policy Gradient Theorem:** a mathematical framework that optimizes neural network policies directly via evaluative trial-and-error, without ever needing an analytical physics model of the environment.

### 1. Likelihood Ratio

The Log-Derivative Trick  
Physics Vanishes!

### 2. Gaussian Policies

Continuous Action Math  
Pushing the Mean

### 3. The Variance Trap

Lucky vs Unlucky Runs  
Noisy Sample Gradients

### 4. Causality Proof

Reward-to-Go Formulation  
Discarding the Past

### 5. Baselines & EGLP

EGLP Lemma Zero Bias  
Optimal Baseline  $b^*$

## Part 1: The Policy Gradient Theorem & The Log-Derivative Trick

**(Slides 1–15)** Let a trajectory be  $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$  with cumulative return  $R(\tau) = \sum_{t=0}^T r(s_t, a_t)$ . The expected return objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\tau)}[R(\tau)] = \int p_\theta(\tau) R(\tau) d\tau$$

### 1.1 Step-by-Step Derivation of the Policy Gradient

We wish to compute  $\nabla_\theta J(\theta)$ . Differentiating directly under the integral sign:

$$\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau) R(\tau) d\tau = \int \nabla_\theta p_\theta(\tau) R(\tau) d\tau$$

**The Obstacle:** We cannot evaluate this integral as an expectation because it is weighted by  $\nabla_\theta p_\theta(\tau)$ , which is *not* a valid probability distribution (it does not integrate to 1 and can be negative!).

**The Solution (The Log-Derivative / Likelihood Ratio Identity):** Using the elementary calculus identity  $\frac{d}{dx} \log f(x) = \frac{f'(x)}{f(x)} \implies f'(x) = f(x) \frac{d}{dx} \log f(x)$ :

$$\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \frac{\nabla_\theta p_\theta(\tau)}{p_\theta(\tau)} = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$$

Substitute this back into the gradient integral:

$$\nabla_{\theta} J(\theta) = \int p_{\theta}(\tau) [\nabla_{\theta} \log p_{\theta}(\tau)] R(\tau) d\tau = \mathbb{E}_{\tau \sim p_{\theta}(\tau)} [\nabla_{\theta} \log p_{\theta}(\tau) R(\tau)]$$

## 1.2 Why the Unknown Physics Dynamics Vanish Identically

Recall the Markov chain expansion of trajectory probability  $p_{\theta}(\tau)$ :

$$p_{\theta}(\tau) = \rho_0(s_0) \prod_{t=0}^{T-1} \pi_{\theta}(a_t | s_t) \mathcal{P}(s_{t+1} | s_t, a_t)$$

Taking the natural logarithm:

$$\log p_{\theta}(\tau) = \log \rho_0(s_0) + \sum_{t=0}^{T-1} \log \pi_{\theta}(a_t | s_t) + \sum_{t=0}^{T-1} \log \mathcal{P}(s_{t+1} | s_t, a_t)$$

Now take the gradient with respect to  $\theta$ . Because initial state distribution  $\rho_0(s_0)$  and transition physics  $\mathcal{P}(s_{t+1} | s_t, a_t)$  do not depend on neural network weights  $\theta$ :

$$\nabla_{\theta} \log \rho_0(s_0) = 0, \quad \nabla_{\theta} \log \mathcal{P}(s_{t+1} | s_t, a_t) = 0$$

### The REINFORCE Policy Gradient Estimator (Williams, 1992):

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \left( \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \right) \left( \sum_{t=0}^{T-1} r(s_t, a_t) \right) \right]$$

### THE PLAIN-ENGLISH INTUITION OF REINFORCE

$\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)$  represents the direction in parameter space that increases the probability of taking action  $a_t$ . Multiplying by scalar return  $R(\tau)$  scales this push:

- If  $R(\tau) > 0$ , the gradient steps to make all actions taken in that trajectory **more likely**.
- If  $R(\tau) < 0$ , the gradient steps to make all actions taken in that trajectory **less likely**.

This is mathematically formal trial-and-error discovery without knowing the laws of physics!

## Part 2: Continuous Actions & Gaussian Policy Gradients

(Slides 16–22) In robotic systems, actions are continuous joint torques or velocities  $a \in \mathbb{R}^{d_a}$ . We model the policy as a Gaussian  $\pi_{\theta}(a | s) = \mathcal{N}(\mu_{\theta}(s), \Sigma_{\theta}(s))$  with diagonal covariance  $\Sigma = \text{diag}(\sigma_1^2, \dots, \sigma_{d_a}^2)$ .

### 2.1 The Analytical Score Function of a Gaussian Policy

For a 1D continuous action with mean  $\mu_{\theta}(s)$  and standard deviation  $\sigma$ :

$$\log \pi_{\theta}(a | s) = -\frac{1}{2} \left[ \frac{(a - \mu_{\theta}(s))^2}{\sigma^2} + 2 \log \sigma + \log(2\pi) \right]$$

Differentiating with respect to network parameters  $\theta$ :

$$\nabla_{\theta} \log \pi_{\theta}(a | s) = \frac{a - \mu_{\theta}(s)}{\sigma^2} \nabla_{\theta} \mu_{\theta}(s)$$

### PHYSICAL ROBOTICS MEANING: PUSHING THE MEAN

Notice the error term  $(a - \mu_{\theta}(s))$ :

- Suppose the policy network intended to push the knife downward at  $\mu = 10$  mm/s.
- Exploratory Gaussian noise sampled  $a = 15$  mm/s, so  $(a - \mu) = +5$  mm/s.
- If this action successfully initiated a clean cut through the tissue cuticle without crushing ( $R(\tau) = +50$ ), the gradient update pushes  $\mu_{\theta}(s)$  toward  $+5$  mm/s, permanently shifting future mean knife speed higher!

### SPINNING UP BUG ALERT: CLAMPING LOG-STD

Joshua Achiam warns: if you let ``log_std`` float unconstrained, gradients will cause  $\sigma \rightarrow 0$  (freezing exploration prematurely) or  $\sigma \rightarrow \infty$  (causing ``NaN`` in division by  $\sigma^2$ ). In production robotics code, always clamp log standard deviations: ``log_std = torch.clamp(log_std, min=-20, max=2)``.

## Part 3: The High Variance Nightmare

(Slides 23–28) While raw REINFORCE is unbiased, it is notoriously impractical in complex environments due to **destructive sample variance**.

### 3.1 The Credit Assignment Flaw: Lucky vs. Unlucky Trajectories

In raw REINFORCE, the sum of all log-probabilities  $\sum_{t=0}^T \nabla \log \pi(a_t | s_t)$  is multiplied by the **entire episode return**  $R(\tau)$ .

#### Trajectory 1: Unlucky Slip at t=195

Steps 0–190: Flawless surgical incision  
 Step 195: Robot slips → Negative total score!  
**REINFORCE PENALIZES ALL 195 STEPS!**

#### Trajectory 2: Lucky Bounce at t=195

Steps 0–190: Erroneous, shaky control  
 Step 195: Random bounce hits target → High score!  
**REINFORCE REWARDS ALL ERRATIC STEPS!**

Because every step is credited with the total score of the entire trajectory, millions of rollouts are needed for the noise to cancel out. We require two fundamental variance reduction techniques: **Causality** and **Baselines**.

## Part 4: Variance Reduction I — Causality & Reward-to-Go

(Slides 29–35) The physical universe obeys causality: **an action taken at time  $t$  cannot retroactively cause or influence rewards received at earlier times  $t' < t$** .

### MATHEMATICAL PROOF: PAST REWARDS HAVE ZERO EXPECTED GRADIENT

Consider the expectation of an action gradient at time  $t$  multiplied by a prior reward  $r_{t'}$  where  $t' < t$ :

$$\mathbb{E}_{\tau} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) r(s_{t'}, a_{t'})]$$

Using the tower property of conditional expectation:

$$= \mathbb{E}_{s_0, a_0, \dots, s_t} [r(s_{t'}, a_{t'}) \mathbb{E}_{a_t \sim \pi_{\theta}(\cdot | s_t)} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) | s_t]]$$

Evaluating the inner expectation over  $a_t$ :

$$\mathbb{E}_{a_t}[\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = \int \pi_{\theta}(a_t | s_t) \frac{\nabla_{\theta} \pi_{\theta}(a_t | s_t)}{\pi_{\theta}(a_t | s_t)} da_t = \nabla_{\theta} \int \pi_{\theta}(a_t | s_t) da_t = \nabla_{\theta}(1) = 0$$

The inner expectation is identically zero!

Therefore, removing past rewards does not introduce any bias, but eliminates massive amounts of uncorrelated noise! This gives the **Reward-to-Go** formulation:

**Reward-to-Go Policy Gradient:**

$$\nabla_{\theta} J(\theta) = \sum_{t=0}^T \mathbb{E}_{(s_t, a_t)} \left[ \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \left( \sum_{t'=t}^T r(s_{t'}, a_{t'}) \right) \right]$$

## Part 5: Variance Reduction II — Baselines, EGLP Lemma & Optimal Baseline

(Slides 36–45 & Spinning Up ch09) Even with reward-to-go, if all rewards are positive ( $r_t \in [100, 110]$ ), the gradient will push *all* sampled actions to become more probable! To differentiate good actions from merely average actions, we subtract a **baseline**  $b(s_t)$ .

### 5.1 The Expected Grad-Log-Prob (EGLP) Lemma

**The Expected Grad-Log-Prob (EGLP) Lemma:**

$$\mathbb{E}_{x \sim P_{\theta}} [\nabla_{\theta} \log P_{\theta}(x)] = 0$$

Because  $\mathbb{E}_{a_t \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)] = 0$ , any baseline function  $b(s_t)$  that does not depend on the current action  $a_t$  leaves the expectation completely unchanged:

$$\mathbb{E} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t) b(s_t)] = \mathbb{E}_{s_t} \left[ b(s_t) \underbrace{\mathbb{E}_{a_t \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a_t | s_t)]}_{=0} \right] = 0$$

### 5.2 The Optimal Baseline Minimizing Variance

What choice of baseline  $b(s)$  minimizes the variance of the gradient estimator  $\text{Var}(\hat{g})$ ? Taking the derivative of variance with respect to  $b$  and setting it to zero yields:

$$b^*(s) = \frac{\mathbb{E}_{a \sim \pi} [\|\nabla_{\theta} \log \pi_{\theta}(a | s)\|^2 Q(s, a)]}{\mathbb{E}_{a \sim \pi} [\|\nabla_{\theta} \log \pi_{\theta}(a | s)\|^2]}$$

This is an expected return weighted by gradient magnitude. In practice, the state value function  $V^{\pi}(s) = \mathbb{E}[Q(s, a)]$  is very close to  $b^*(s)$  and is far easier to train via a neural network critic!

### 5.3 Achiam's 5 Forms of $\Phi_t$ (Spinning Up Universal Formulation)

Joshua Achiam showed that all policy gradient methods can be written as:

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau} \left[ \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \Phi_t \right]$$

| Choice of $\Phi_t$      | Mathematical Expression                                   | Bias & Variance Characterization                                             |
|-------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------|
| 1. Total Return         | $\Phi_t = R(\tau) = \sum_{t'=0}^T r_{t'}$                 | Unbiased; <b>Maximum Variance</b> (Classic REINFORCE).                       |
| 2. Reward-to-Go         | $\Phi_t = \sum_{t'=t}^T r_{t'}$                           | Unbiased; <b>Lower Variance</b> (Removes past uncorrelated noise).           |
| 3. Return with Baseline | $\Phi_t = \sum_{t'=t}^T r_{t'} - b(s_t)$                  | Unbiased; <b>Significantly Lower Variance</b> (Centers returns around mean). |
| 4. Q-Function           | $\Phi_t = Q^\pi(s_t, a_t)$                                | Biased (if approximated by neural network); <b>Very Low Variance</b> .       |
| 5. Advantage Function   | $\Phi_t = A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t)$ | Biased (if approximated); <b>Lowest Variance</b> (PPO / Actor-Critic).       |

## Part 6: Formal Algorithm Specification & PyTorch Implementation

### ALGORITHM 1: REINFORCE WITH LEARNED STATE-VALUE BASELINE

**Initialize:** Policy network parameters  $\theta$ , Baseline value network parameters  $\phi$ , learning rates  $\alpha_\theta, \alpha_\phi$ .

1. **for** iteration  $k = 1, 2, \dots$  **do**
2. Sample batch of trajectories  $\mathcal{D} = \{\tau_i\}$  by running policy  $\pi_\theta$  in environment.
3. **for** each trajectory  $\tau_i$  and each time step  $t$  **do**
4. Compute reward-to-go:  $\hat{R}_{i,t} = \sum_{t'=t}^T \gamma^{t'-t} r_{i,t'}$ .
5. Compute advantage estimate:  $\hat{A}_{i,t} = \hat{R}_{i,t} - V_\phi(s_{i,t})$ .
6. **end for**
7. Normalize advantages across batch:  $\hat{A} \leftarrow \frac{\hat{A} - \text{mean}(\hat{A})}{\text{std}(\hat{A}) + 10^{-8}}$ .
8. Update Baseline Critic by minimizing MSE loss:

$$\phi \leftarrow \phi - \alpha_\phi \nabla_\phi \frac{1}{|\mathcal{D}|} \sum_{i,t} (V_\phi(s_{i,t}) - \hat{R}_{i,t})^2$$

9. Update Policy Actor via policy gradient ascent:

$$\theta \leftarrow \theta + \alpha_\theta \frac{1}{|\mathcal{D}|} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_{i,t} | s_{i,t}) \hat{A}_{i,t}$$

10. **end for**

## 6.1 Production PyTorch Implementation

```
import torch
import torch.nn as nn
from torch.distributions.normal import Normal

class ContinuousActorCritic(nn.Module):
    def __init__(self, obs_dim: int, act_dim: int):
        super().__init__()
        # Policy network (Actor)
        self.actor_backbone = nn.Sequential(
            nn.Linear(obs_dim, 128), nn.Tanh(),
            nn.Linear(128, 128), nn.Tanh()
        )
        self.mu_head = nn.Linear(128, act_dim)
        self.log_std = nn.Parameter(torch.zeros(act_dim))

        # Value network (Baseline Critic)
        self.critic = nn.Sequential(
            nn.Linear(obs_dim, 128), nn.Tanh(),
            nn.Linear(128, 128), nn.Tanh(),
            nn.Linear(128, 1)
        )

    def get_action_and_value(self, obs: torch.Tensor, action: torch.Tensor = None):
        features = self.actor_backbone(obs)
        mu = self.mu_head(features)
        std = torch.exp(torch.clamp(self.log_std, min=-20.0, max=2.0))
        dist = Normal(mu, std)

        if action is None:
            action = dist.sample()

        # Sum log probabilities over action dimensions!
        log_prob = dist.log_prob(action).sum(dim=-1, keepdim=True)
        entropy = dist.entropy().sum(dim=-1, keepdim=True)
        value = self.critic(obs)
        return action, log_prob, entropy, value
```

## Part 7: Conceptual Mastery & Exam Challenge

**Question 1: If we multiply all environment rewards by 10 (scaling  $r \rightarrow 10r$ ), does the policy gradient update change?**

**Answer:** In raw REINFORCE without advantage normalization, YES: the gradient magnitude scales by 10, effectively increasing learning rate by 10x and potentially causing catastrophic optimization instability. However, if **batch advantage normalization** is used ( $\hat{A} \leftarrow (\hat{A} - \mu)/\sigma$ ), the constant factor 10 cancels out, rendering the policy gradient invariant to reward scaling!

**Question 2: Why must the baseline  $V_\phi(s)$  be detached (`.detach()`) when computing the policy gradient loss in PyTorch?**

**Answer:** The advantage is  $\hat{A}_t = R_t - V_\phi(s_t)$ . When we write `loss = -(log_prob * advantage).mean()`, if  $V_\phi$  is not detached, backpropagation will compute  $\nabla_\phi$  through the advantage term, trying to minimize advantage rather than maximize return! Detaching ensures the policy optimizer updates only the Actor parameters  $\theta$ .

**Question 3: Why is policy gradient loss often negative and continuously decreasing, even if the robot's task performance is degrading?**

**Answer:** The policy gradient loss  $-\sum \log \pi_\theta(a|s) \hat{A}$  is a *surrogate loss* evaluated on non-stationary data collected by an older policy. It is not an upper bound on performance. If entropy decays and  $\log \pi$  explodes, the numerical loss will decrease to  $-\infty$  even as the policy collapses to a single catastrophic action.